



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Name: M. Samanvi

Reg. No.: 192425318

Course Code: MLA0401

Course Name: Deep Learning

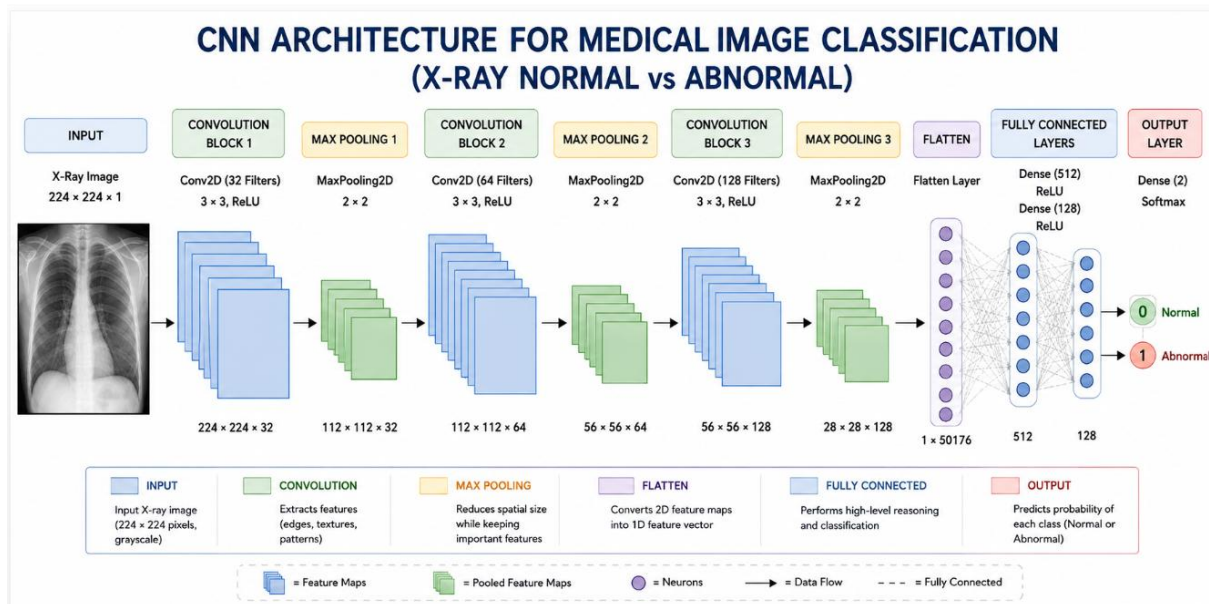
Question 1: Medical Image Classification using CNN

Workflow

1. **Input X-ray**
 - Take an X-ray image as input.
2. **Preprocessing**
 - Resize the image.
 - Normalize pixel values.
3. **Convolution Layer**
 - Detect edges, shapes and textures.
4. **ReLU Activation**
 - Adds non-linearity and helps the network learn complex patterns.
5. **Max Pooling**
 - Reduces image size and keeps important features.
6. **More Convolution + Pooling**
 - Learns deeper features such as medical patterns.
7. **Flatten**
 - Converts feature maps into a 1D vector.
8. **Fully Connected Layers**
 - Combine the extracted features for classification.
9. **Softmax Output**
 - Produces probabilities for the two classes.

10. Final Result

- 0 → Normal
- 1 → Abnormal



Datasets

```
xray_normal_abnormal_dataset.csv
1 pixel_0,pixel_1,pixel_2,pixel_3,pixel_4,pixel_5,pixel_6,pixel_7,pixel_8,pixel_9,pixel_10,pixel_11,pixel_12,pixel_13,pixel_14,pixel_15,pixel_16,pixel_17,pixel_18,pixel_19
2 0.0849641728687641,0.0654864560532546,0.10476823457212858,0.14860571064670303,0.08804570220237716,0.098175929115578,0.18064145853415658,0.15730138484738512,0.1156589225
3 0.08748722466555639,0.1145104667435774,0.11221757923065704,0.10605171955951949,0.09460528873296471,0.041102970052905564,0.13465767467916434,0.1349115031677932,0.1453010
4 0.0805937724728530893,0.06011711855832107,0.11063128023207397,0.03751293946361204,0.1015415447797576,0.053278135208609016,0.13613014521120198,0.12517831638936943,0.0698
5 0.092438796763139,0.016640989204151406,0.1271304524615569,0.09813453763972627,0.08264075162145246,0.1127684680722526,0.04642353659792812,0.1429501081404956,0.091263037
6 0.103157597512765,0.03043271924076883,0.08779956080951802,0.07697887275443029,0.131262681999703,0.0224523105704854,0.11359917380421816,0.10249116044991032,0.151728416
7 0.13931573934088532,0.08024860743158453,0.0650036328078249,0.0212218220262489,0.086904563786638,0.09408592957709978,0.1558851776511972,0.14510334064304295,0.087261333
8 0.08819014775258126,0.06613283978809062,0.1351390828837709,0.06831977668698032,0.135622666795988,0.1059916915939635,0.09313891532668742,0.15010520797397923,0.1508232427
9 0.03663711898496661,0.0559000424930892,0.06241969558797296,0.1285484355338355,0.08226144372718687,0.19084782027793712,0.1347074426744624,0.07557668928143953,0.12926046
10 0.06141686810019244,0.028002642903647543,0.0627291817166333,0.091065595741755,0.13382046853790042,0.1002802805072591,0.0691049746427667,0.1288736630114154,0.13897613
11 0.05428093877868015,0.070183848360231,0.15698498097900385,0.11516278167432911,0.15895428445969174,0.13144291562933433,0.08902081683588174,0.13128896432726284,0.1438868
12 0.06257985207232047,0.05336411185489453,0.10518641448820341,0.07187317446973479,0.16113040103240583,0.16614982753280044,0.08181120631968072,0.129392166454843998,0.1084116
13 0.10597411781544046,0.07972759362115113,0.06527948678146142,0.12612180353471247,0.12328801812157322,0.08016379131009792,0.06436099897468368,0.11506279653909694,0.1950544
14 0.049519464287299746,0.12551431122943292,0.07746303302975557,0.05139145401549139,0.11975964850818335,0.09135586659190738,0.06849745497954274,0.09089306339022275,0.203799
15 0.10818735366894251,0.07056473145666352,0.06547653886688906,0.11030983246585083,0.10831255680049431,0.14875895881901688,0.16913170147647544,0.13888411808148324,0.1461072
16 0.09519494183128353,0.04775487369520048,0.12743158292534484,0.10296586461902245,0.00371270368466712,0.11952878648706228,0.1012317797373523,0.1480911882629382,0.17220649
17 0.06782387805242826,0.1039890016224632,0.009353245118287137,0.0910827461120241,0.094609979442192,0.13655816047985586,0.1099648816788994,0.08505461831476198,0.13473192
18 0.0390638605455764,0.040952664872175734,0.06970931929132562,0.09439217778476447,0.0847599757367168,0.1538808832719728,0.1377964544906924,0.06297016140692095,0.155387008
19 0.06731148321964817,0.08868115251779064,0.045724564178178835,0.05747954045244128,0.1169656727838381,0.202925319406804,0.15760547842645697,0.1772249125380104,0.1267578472
20 0.06556402793636955,0.051681456813214365,0.0,0.10312550113872442,0.11915880232253487,0.17161735905714445,0.08594057532286237,0.0917974693708727,0.027238518587110745,0.1
21 0.03937254684464968,0.045211051723443516,0.07349455996840468,0.05278073533413589,0.07547596567886329,0.118190017799615,0.09740707876208261,0.116835388974024202,0.0758496
22 0.03428652742390912,0.09302019095417398,0.10612711910399845,0.1829000262369616,0.14243903447624906,0.11726552972668179,0.092662374804715377,0.13835388974822874,0.14750173
23 0.0327051808431751,0.03906315084954964,0.0796715687329924,0.02639564490076956,0.0866221292684429,0.11765647369417191,0.1888583701918518,0.14271272325206164,0.159447036
24 0.0259897326212258,0.11243431531933061,0.006527010887131518,0.0821039552570592,0.12181717899482797,0.1212261841979315,0.08878162699323416,0.15301434394953076,0.09899632
25 0.0393914028360289,0.05131282262363158,0.11736498401116696,0.07353181534189478,0.048137089665987384,0.13071335319673066,0.1546236473168919,0.1090808497512731,0.163262
26 0.1276023469575421,0.0904081752586778,0.06535073174002348,0.05575708634583374,0.0681205407741025,0.0854107010460748,0.20351905757089168,0.1988552875657272,0.1642890336
27 0.0545163927975699,0.049527153837182,0.07246300711614752,0.06565728252411152,0.09220401587912012,0.1081990445383205,0.08155571017865863,0.15183119016518398,0.1196207894
28 0.04033650401203099,0.053083190927967525,0.09838686362965382,0.06704259014631088,0.11927281383345112,0.11135196996861384,0.1141076258789146,0.0636939745360923,0.101648
29 0.1123083203154198,0.021358794096426454,0.09885236226797928,0.14503451775895143,0.11779836149971552,0.12667779472729626,0.09860724276845456,0.13540838863301533,0.1838906
30 0.0651284602194932,0.07244656797438938,0.08716182416263779,0.03822923125412377,0.12240865575540488,0.05733348279691199,0.0798832266269985,0.08463540492098219,0.12742889
31 0.03957308637566961,0.1230870647280359,0.07936520642190858,0.036915655176251205,0.08205342910991699,0.1126496848166277,0.1538515717167395,0.1024129068665805,0.1152968
32 0.10001507400201756,0.03751072884778095,0.02346675389874335,0.07108396431851921,0.113057883332071,0.13142942212108027,0.1718385324594166,0.08029215667650879,0.116871
33 0.0854621139513817,0.0808568272860783,0.142135938038421,0.07398928217637803,0.08265521113224604,0.05795517102483051,0.1097167396619696,0.147893581552945,0.1640383225
34 0.029043618445902235,0.11014128609001133,0.07592760257367616,0.1078582837555486,0.1204413162811019,0.10974044225601663,0.09765996949211275,0.1322403011108074,0.145118
35 0.0758875697306343,0.04872442125105953,0.03440019303180954,0.07609892068329003,0.10474126115271422,0.11752840851305661,0.15971393571947162,0.17889636044041388,0.14789013
36 0.08637160178783311,0.0530437662705030,0.036120187767301487,0.057490174767230802,0.1668034715568458,0.08175065050367878,0.1125387216087846,0.10677276118002816,0.085311
```

```
# =====
# CNN FOR X-RAY MEDICAL IMAGE CLASSIFICATION
# =====
```

!pip -q install tensorflow

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from google.colab import files

from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D
from tensorflow.keras.layers import Flatten, Dense

# =====
# 1. UPLOAD DATASET
# =====

uploaded = files.upload()

# =====
# 2. LOAD DATASET
# =====

df = pd.read_csv("xray_normal_abnormal_dataset.csv")

print("Dataset Shape:", df.shape)
print(df.head())

# =====
# 3. SEPARATE FEATURES AND LABEL
# =====

X = df.iloc[:, :-1].values
y = df.iloc[:, -1].values

# =====
# 4. RESHAPE IMAGE
# =====

X = X.reshape(-1, 28, 28, 1)

# =====
# 5. TRAIN TEST SPLIT
# =====

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

```

```
# =====  
# 6. BUILD CNN  
# =====
```

```
model = Sequential()
```

```
# Convolution Block 1
```

```
model.add(  
    Conv2D(  
        32,  
        (3,3),  
        activation="relu",  
        input_shape=(28,28,1)  
    )  
)
```

```
model.add(MaxPooling2D((2,2)))
```

```
# Convolution Block 2
```

```
model.add(  
    Conv2D(  
        64,  
        (3,3),  
        activation="relu"  
    )  
)
```

```
model.add(MaxPooling2D((2,2)))
```

```
# Convolution Block 3
```

```
model.add(  
    Conv2D(  
        128,  
        (3,3),  
        activation="relu"  
    )  
)
```

```
model.add(MaxPooling2D((2,2)))
```

```
# =====  
# 7. FLATTEN  
# =====
```

```
model.add(Flatten())
```

```
# =====  
# 8. FULLY CONNECTED LAYERS  
# =====
```

```
model.add(  
    Dense(  
        128,  
        activation="relu"  
    )  
)
```

```

)

model.add(
    Dense(
        64,
        activation="relu"
    )
)

# =====
# 9. OUTPUT LAYER
# =====

model.add(
    Dense(
        2,
        activation="softmax"
    )
)

# =====
# 10. COMPILE
# =====

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

# Display architecture
model.summary()

# =====
# 11. TRAIN MODEL
# =====

history = model.fit(
    X_train,
    y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.2
)

# =====
# 12. TEST MODEL
# =====

loss, accuracy = model.evaluate(
    X_test,
    y_test
)

print("\nTest Accuracy:",

```

```

        round(accuracy * 100, 2), "%")

print("Test Loss:",
      round(loss, 4))

# =====
# 13. ACCURACY GRAPH
# =====

plt.figure(figsize=(8,5))

plt.plot(
    history.history["accuracy"],
    label="Training Accuracy"
)

plt.plot(
    history.history["val_accuracy"],
    label="Validation Accuracy"
)

plt.title("CNN Training and Validation Accuracy")

plt.xlabel("Epoch")
plt.ylabel("Accuracy")

plt.legend()
plt.grid()

plt.show()

# =====
# 14. LOSS GRAPH
# =====

plt.figure(figsize=(8,5))

plt.plot(
    history.history["loss"],
    label="Training Loss"
)

plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
)

plt.title("CNN Training and Validation Loss")

plt.xlabel("Epoch")
plt.ylabel("Loss")

plt.legend()
plt.grid()

```

```

plt.show()

# =====
# 15. PREDICTION
# =====

predictions = model.predict(X_test)

predicted_classes = np.argmax(
    predictions,
    axis=1
)

# =====
# 16. CONFUSION MATRIX
# =====

cm = confusion_matrix(
    y_test,
    predicted_classes
)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Normal", "Abnormal"]
)

disp.plot()

plt.title("CNN Confusion Matrix")

plt.show()

# =====
# 17. SAMPLE PREDICTION
# =====

sample = X_test[0]

prediction = model.predict(
    sample.reshape(1,28,28,1)
)

result = np.argmax(prediction)

if result == 0:
    print("Prediction: NORMAL X-RAY")
else:
    print("Prediction: ABNORMAL X-RAY")

```

Output:

Choose Files xray_normal...dataset.csv

xray_normal_abnormal_dataset.csv(text/csv) - 15286512 bytes, last modified: 8/10/2026 - 100% done

*** Saving xray_normal_abnormal_dataset.csv to xray_normal_abnormal_dataset.csv
Dataset Shape: (1000, 785)

```
pixel_0 pixel_1 pixel_2 pixel_3 pixel_4 pixel_5 pixel_6 \
0 0.084964 0.065749 0.104768 0.148606 0.088046 0.098176 0.180641
1 0.087487 0.114510 0.112218 0.106052 0.094605 0.041103 0.134658
2 0.002594 0.050117 0.110631 0.037513 0.109154 0.053278 0.136130
3 0.092438 0.016641 0.127338 0.098135 0.082641 0.113277 0.046424
4 0.103158 0.030433 0.097800 0.076979 0.131283 0.022452 0.113509
```

```
pixel_7 pixel_8 pixel_9 ... pixel_775 pixel_776 pixel_777 \
0 0.157301 0.115659 0.162375 ... 0.091458 0.115508 0.123417
1 0.134912 0.145301 0.089602 ... 0.143422 0.143739 0.103429
2 0.125178 0.069833 0.187262 ... 0.133225 0.058460 0.177204
3 0.142950 0.093263 0.086566 ... 0.206999 0.147501 0.116858
4 0.102491 0.151728 0.159474 ... 0.163008 0.184972 0.099520
```

```
pixel_778 pixel_779 pixel_780 pixel_781 pixel_782 pixel_783 label
0 0.118077 0.133373 0.103642 0.032224 0.091484 0.093064 0.0
1 0.024085 0.143908 0.126008 0.074385 0.078956 0.089278 0.0
2 0.101901 0.074035 0.124273 0.069958 0.118674 0.093321 0.0
3 0.117485 0.107043 0.027286 0.145436 0.077929 0.086361 0.0
4 0.061094 0.104872 0.093037 0.107571 0.114260 0.066264 0.0
```

[5 rows x 785 columns]

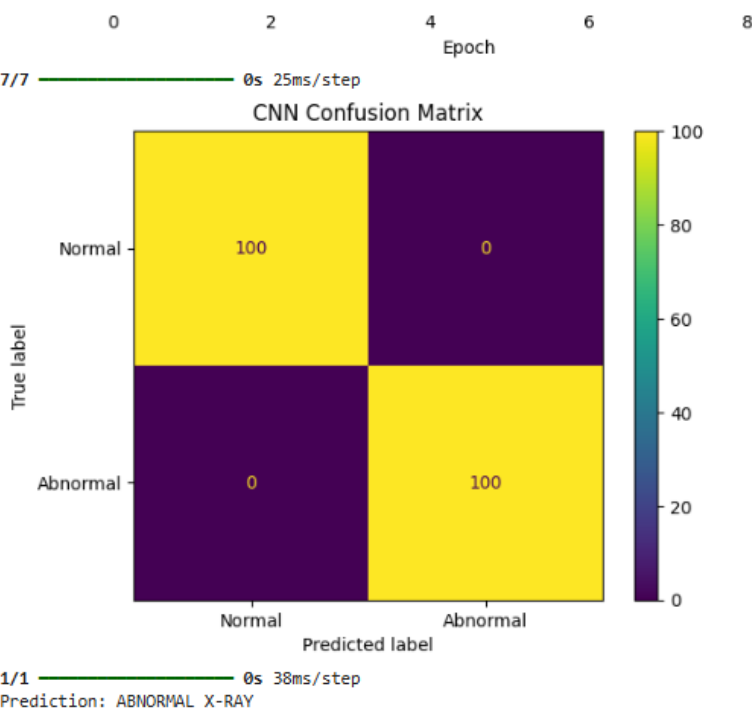
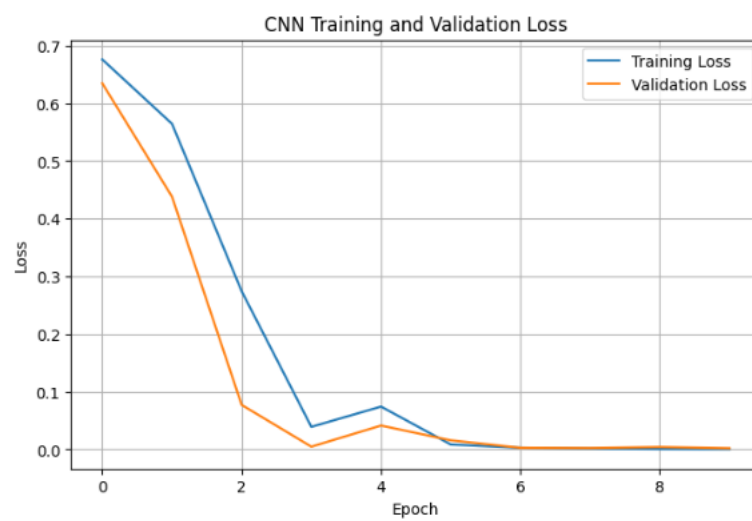
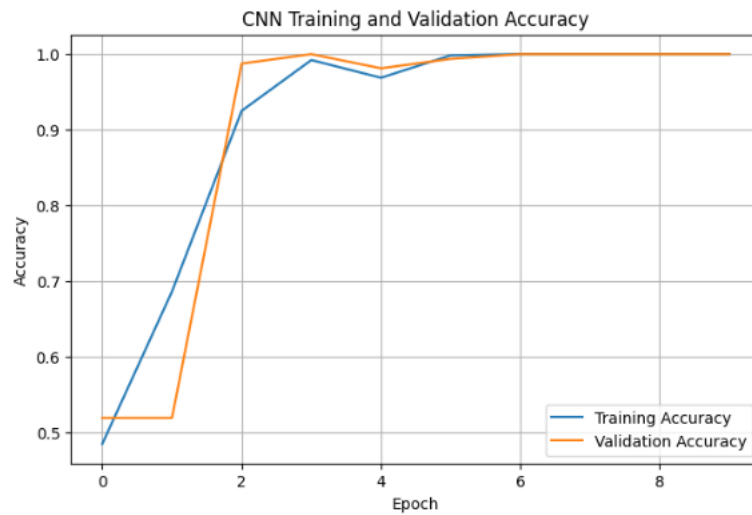
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input\_shape`/`input\_c`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
conv2d_2 (Conv2D)	(None, 3, 3, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 1, 1, 128)	0
flatten (Flatten)	(None, 128)	0
dense (Dense)	(None, 128)	16,512
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 2)	130

Total params: 117,570 (459.26 KB)

Trainable params: 117,570 (459.26 KB)



Question 2: Face Recognition System using CNN

Workflow

Face Image from Camera



Image Preprocessing

(Resize + Normalize)



Convolution Layer 1

(32 Filters + ReLU)



Max Pooling



Convolution Layer 2

(64 Filters + ReLU)



Max Pooling



Convolution Layer 3

(128 Filters + ReLU)



Max Pooling



Convolution Layer 4

(256 Filters + ReLU)



Flatten



Fully Connected Layer

(512 Neurons + ReLU)



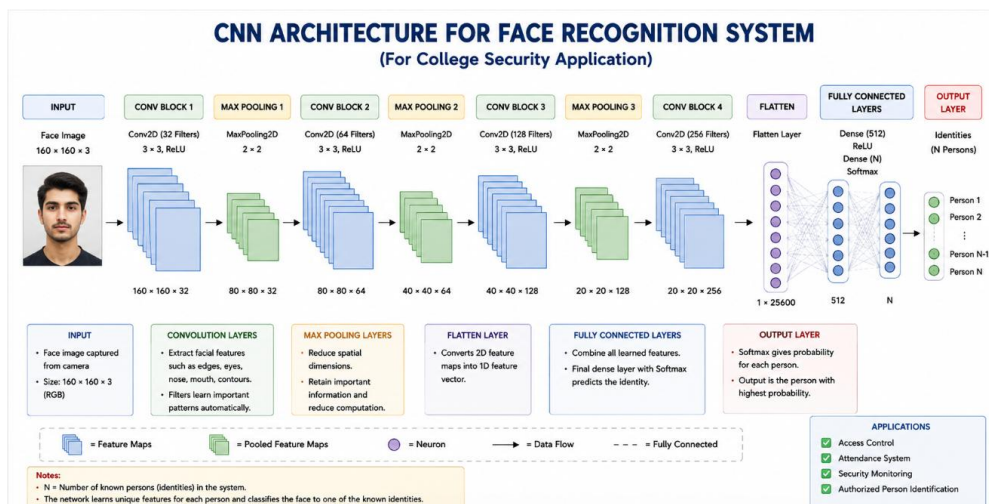
Output Layer

(Softmax)



Person Identification

(Person 1 / Person 2 / ... / Person N)



Dataset

```
face_recognition_features.csv
1 EyeDistance,NoseWidth,MouthWidth,FaceHeight,JawWidth,ForeheadHeight,LeftEyeSize,RightEyeSize,SmileScore,FaceClass
2 64.48,31.41,51.87,191.75,137.03,61.74,26.66,31.53,0.05,1
3 61.31,33.91,49.99,161.21,130.31,61.42,26.57,29.36,0.2,3
4 65.24,34.05,51.31,176.72,122.7,55.32,30.85,29.35,0.29,9
5 69.62,35.42,54.86,179.58,126.3,62.9,31.73,26.21,0.59,8
6 60.83,29.9,57.11,180.16,132.74,52.55,26.14,31.54,0.81,7
7 60.83,35.78,54.28,181.8,144.55,56.73,26.66,30.75,0.06,9
8 69.9,25.89,52.41,186.1,126.13,50.0,30.29,26.31,0.25,7
9 65.84,32.11,57.99,179.65,136.44,67.79,29.07,27.05,0.38,6
10 59.65,32.54,50.75,176.32,127.38,58.84,29.35,30.51,0.18,5
11 64.71,35.9,56.12,175.92,130.88,70.83,29.79,31.59,0.55,6
12 59.68,34.07,50.85,173.88,142.12,64.01,27.52,25.47,0.48,5
13 59.67,29.78,53.72,187.88,136.11,62.24,28.66,25.99,0.32,2
14 63.21,31.05,51.55,173.53,120.07,57.07,28.5,27.71,0.52,5
15 52.43,34.6,51.78,188.47,125.42,54.77,26.28,23.66,0.36,6
16 53.38,33.76,50.49,180.19,140.05,62.61,28.63,28.04,0.34,5
17 59.19,32.26,57.38,174.79,131.14,59.64,30.48,30.37,0.27,1
18 56.94,32.56,49.07,186.18,120.81,53.3,27.33,25.12,0.54,5
19 63.57,36.09,49.04,172.73,123.45,52.42,28.31,28.25,0.44,1
20 57.46,32.45,60.48,186.38,135.63,55.21,28.41,26.84,0.2,0
21 54.94,33.74,55.91,182.13,137.96,69.1,27.43,31.28,0.01,9
22 69.33,40.51,42.46,174.04,130.14,71.07,29.35,26.86,0.15,9
23 60.87,29.73,49.74,185.29,149.8,61.02,26.64,31.04,0.04,9
24 62.34,28.44,44.75,185.47,136.59,60.74,30.67,26.81,0.43,4
25 54.88,36.35,48.2,191.58,128.05,58.32,29.25,29.54,0.23,3
26 59.28,31.96,39.45,189.66,123.47,55.33,26.13,28.5,0.42,8
27 62.55,33.05,51.45,184.91,122.2,66.88,31.4,27.72,0.33,8
28 56.25,31.66,47.43,168.72,132.68,69.37,28.34,26.42,0.54,8
29 63.88,33.22,52.24,204.49,126.09,55.53,26.55,28.65,0.72,2
30 59.0,38.8,45.5,165.96,143.39,59.59,30.21,27.12,0.48,9
31 60.54,36.41,52.33,189.49,141.73,57.75,28.99,26.62,0.06,2
32 58.99,36.59,50.34,184.31,135.06,57.9,24.46,30.95,0.02,2
33 71.26,29.75,58.01,168.26,120.57,65.93,26.95,29.1,0.21,9
34 61.93,30.55,57.65,178.76,124.62,53.44,28.56,29.29,0.42,4
35 56.71,25.48,55.14,179.21,140.05,62.33,23.68,27.22,0.04,5
36 66.11,31.57,50.60,193.06,125.5,63.64,28.17,27.63,0.33,5
```

Program:

```
# =====
# FACE RECOGNITION USING DEEP NEURAL NETWORK
# =====

!pip -q install tensorflow

# -----
# Import Libraries
# -----

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from google.colab import files

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# -----
# Upload Dataset
# -----

uploaded = files.upload()

# -----
# Read Dataset
# -----

df = pd.read_csv("face_recognition_features.csv")

print("Dataset Shape:", df.shape)

print("\nFirst 5 Rows:")
print(df.head())

# -----
# Separate Features and Label
# -----

X = df.iloc[:, :-1].values
```

```

y = df.iloc[:, -1].values

# -----
# Train-Test Split
# -----

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# -----
# Normalize Features
# -----

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)

# -----
# Build Deep Neural Network
# -----

model = Sequential()

# Input + Hidden Layer 1
model.add(
    Dense(
        64,
        activation="relu",
        input_shape=(X_train.shape[1],)
    )
)

# Hidden Layer 2
model.add(
    Dense(
        128,
        activation="relu"
    )
)

# Hidden Layer 3
model.add(
    Dense(

```

```

        64,
        activation="relu"
    )
)

# Hidden Layer 4
model.add(
    Dense(
        32,
        activation="relu"
    )
)

# Output Layer
model.add(
    Dense(
        10,
        activation="softmax"
    )
)

# -----
# Compile Model
# -----

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

# Show Architecture
model.summary()

# -----
# Train Model
# -----

history = model.fit(
    X_train,
    y_train,
    epochs=30,
    batch_size=32,
    validation_split=0.20,
    verbose=1
)

# -----
# Evaluate Model
# -----

```

```

loss, accuracy = model.evaluate(
    X_test,
    y_test,
    verbose=0
)

print("\nTest Accuracy:",
      round(accuracy * 100, 2), "%")

print("Test Loss:",
      round(loss, 4))

# =====
# GRAPH 1: ACCURACY
# =====

plt.figure(figsize=(8,5))

plt.plot(
    history.history["accuracy"],
    label="Training Accuracy"
)

plt.plot(
    history.history["val_accuracy"],
    label="Validation Accuracy"
)

plt.title("Face Recognition Model Accuracy")

plt.xlabel("Epoch")

plt.ylabel("Accuracy")

plt.legend()

plt.grid()

plt.show()

# =====
# GRAPH 2: LOSS
# =====

plt.figure(figsize=(8,5))

plt.plot(
    history.history["loss"],
    label="Training Loss"

```

```

)

plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
)
plt.title("Face Recognition Model Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid()
plt.show()
predictions = model.predict(X_test)
predicted_classes = np.argmax(
    predictions,
    axis=1
)
# =====
# GRAPH 3: CONFUSION MATRIX
# =====
cm = confusion_matrix(
    y_test,
    predicted_classes
)
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm
)
disp.plot()
plt.title("Face Recognition Confusion Matrix")
plt.show()
# =====
# SAMPLE PREDICTION
# =====

sample = X_test[0].reshape(1, -1)

prediction = model.predict(sample)

person = np.argmax(prediction)

print("\nPredicted Person/Class:", person)
print(
    "Confidence:",
    round(np.max(prediction) * 100, 2),
    "%"
)

```



```
... face_recog_features.csv
face_recognition_features.csv(10000) - 224473 bytes, last modified: 8/10/2026 - 100% done
Saving face_recognition_features.csv to face_recognition_features.csv
Dataset Shape: (4800, 10)

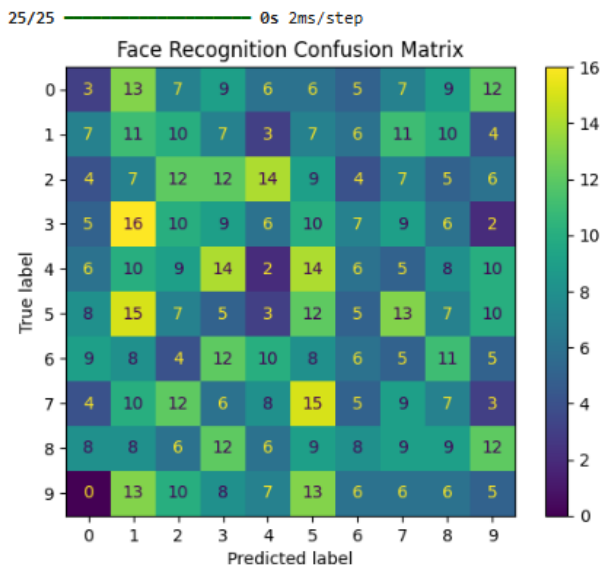
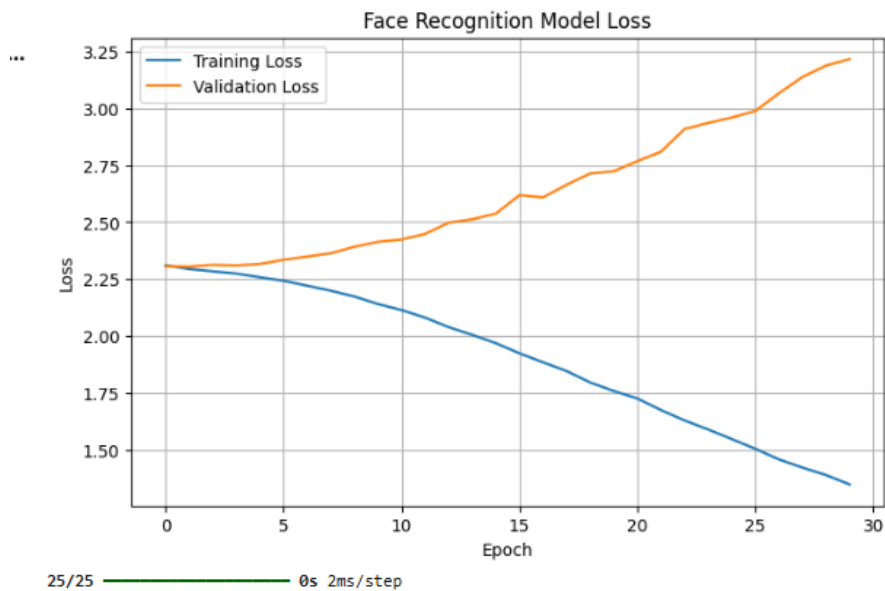
First 5 Rows:
EyeDistance NoseWidth MouthWidth FaceHeight JawWidth ForeheadHeight \
0 64.48 31.41 51.87 191.75 137.03 61.74
1 61.31 33.91 49.99 161.21 130.31 61.42
2 65.14 34.05 51.31 176.72 122.70 55.32
3 69.62 35.42 54.86 179.58 126.30 62.90
4 66.83 29.90 57.11 186.16 132.74 52.55

LeftEyeSize RightEyeSize SmileScore FaceClass
0 26.66 31.53 0.65 1
1 26.57 29.36 0.20 3
2 30.85 29.35 0.29 9
3 31.73 26.21 0.59 8
4 26.14 31.54 0.81 7

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:186: UserWarning: Do not pass an 'input_shape' / 'input_dim' argument to a layer. When using Sequential models, prefer using an 'Input(shape)' object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_1"

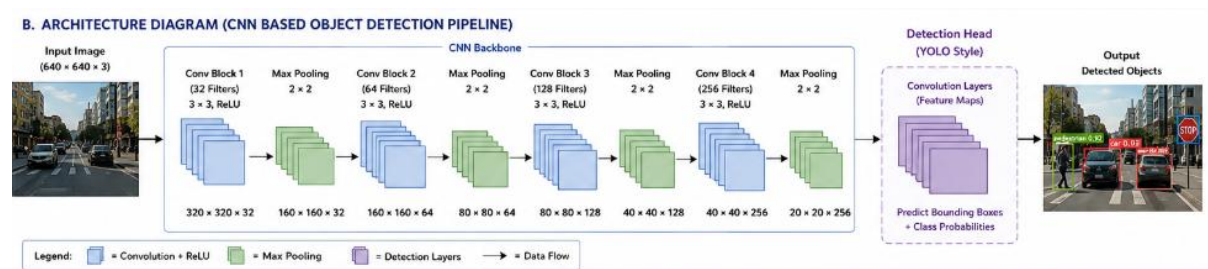
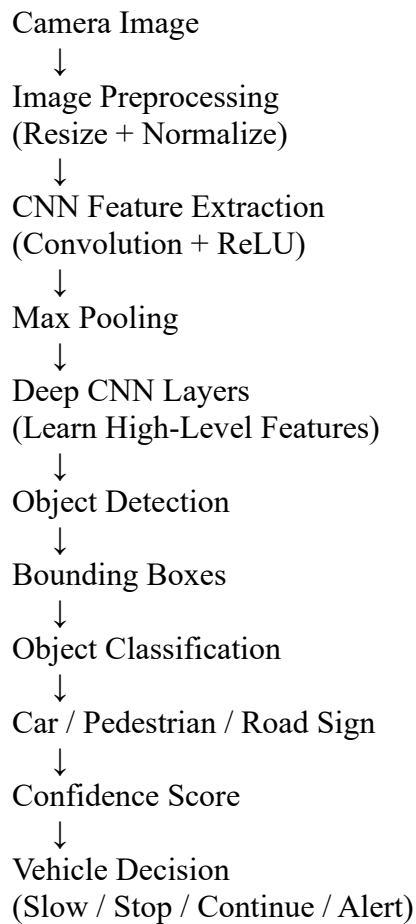
Layer (type) Output Shape Param #
-----
dense_3 (Dense) (None, 64) 640
dense_4 (Dense) (None, 128) 8,320
dense_5 (Dense) (None, 64) 8,256
dense_6 (Dense) (None, 32) 2,080
dense_7 (Dense) (None, 10) 330

Total params: 19,626 (76.66 KB)
Trainable params: 19,626 (76.66 KB)
Non-trainable params: 0 (0.00 KB)
Epoch 1/30 2s 9ms/step - accuracy: 0.0918 - loss: 2.3104 - val_accuracy: 0.0766 - val_loss: 2.3066
Epoch 2/30 0s 3ms/step - accuracy: 0.1289 - loss: 2.2943 - val_accuracy: 0.1203 - val_loss: 2.3051
Epoch 3/30 0s 4ms/step - accuracy: 0.1441 - loss: 2.2841 - val_accuracy: 0.0969 - val_loss: 2.3126
Epoch 4/30 0s 3ms/step - accuracy: 0.1523 - loss: 2.2742 - val_accuracy: 0.1047 - val_loss: 2.3101
Epoch 5/30 0s 3ms/step - accuracy: 0.1602 - loss: 2.2580 - val_accuracy: 0.1125 - val_loss: 2.3162
Epoch 6/30 0s 3ms/step - accuracy: 0.1820 - loss: 2.2427 - val_accuracy: 0.0996 - val_loss: 2.3347
Epoch 7/30 0s 3ms/step - accuracy: 0.1809 - loss: 2.2211 - val_accuracy: 0.0781 - val_loss: 2.3490
Epoch 8/30
```



Autonomous Vehicles — Object Detection using CNN

Workflow



Dataset:

```
xray_normal_abnormal_dataset.csv  autonomous_vehicle_object_detection.csv  face_recognition_features.csv
C: > Users > SAMANVI > Downloads > saveetha clg > DL > CO3 > AT1 > autonomous_vehicle_object_detection.csv
1  image_id,object_id,class,xmin,ymin,xmax,ymax
2  img_0001.jpg,1,pedestrian,270,106,371,196
3  img_0001.jpg,2,pedestrian,121,214,225,318
4  img_0001.jpg,3,road_sign,99,359,152,391
5  img_0002.jpg,1,car,343,293,374,386
6  img_0002.jpg,2,road_sign,160,313,211,450
7  img_0003.jpg,1,car,58,169,179,258
8  img_0004.jpg,1,car,445,174,536,254
9  img_0004.jpg,2,car,243,319,275,449
10 img_0004.jpg,3,pedestrian,20,328,88,375
11 img_0004.jpg,4,road_sign,315,13,458,51
12 img_0005.jpg,1,car,339,91,479,180
13 img_0005.jpg,2,car,263,34,370,144
14 img_0006.jpg,1,car,387,1,422,84
15 img_0006.jpg,2,car,309,190,356,309
16 img_0006.jpg,3,car,201,269,325,346
17 img_0006.jpg,4,car,461,214,552,283
18 img_0007.jpg,1,car,337,366,419,419
19 img_0008.jpg,1,road_sign,187,379,325,449
20 img_0008.jpg,2,car,300,64,418,164
21 img_0009.jpg,1,pedestrian,491,135,608,227
22 img_0010.jpg,1,car,391,162,455,224
23 img_0010.jpg,2,pedestrian,489,230,559,287
24 img_0010.jpg,3,road_sign,327,267,390,329
25 img_0011.jpg,1,car,61,215,127,343
26 img_0011.jpg,2,pedestrian,213,34,307,162
27 img_0011.jpg,3,car,461,130,491,164
28 img_0011.jpg,4,pedestrian,254,397,386,453
29 img_0012.jpg,1,pedestrian,345,41,451,121
30 img_0013.jpg,1,road_sign,240,51,365,84
31 img_0013.jpg,2,road_sign,406,230,544,274
32 img_0013.jpg,3,pedestrian,35,12,96,112
33 img_0014.jpg,1,pedestrian,283,65,354,139
34 img_0014.jpg,2,car,133,283,190,420
35 img_0014.jpg,3,car,285,330,433,451
36 img_0015.jpg,1,car,224,384,280,475
```

Program:

```
# =====
# AUTONOMOUS VEHICLE OBJECT CLASSIFICATION
# USING CNN
# =====
```

```
!pip -q install tensorflow
```

```
# -----
# Import Libraries
# -----
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
from google.colab import files
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
```

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import Dense, Dropout
```

```
# -----  
# Upload Dataset  
# -----
```

```
uploaded = files.upload()
```

```
# -----  
# Load Dataset  
# -----
```

```
df = pd.read_csv(  
    "autonomous_vehicle_object_detection.csv"  
)
```

```
print("Dataset Shape:", df.shape)
```

```
print("\nFirst 5 Rows:")  
print(df.head())
```

```
# -----  
# Prepare Features  
# -----
```

```
X = df[  
    ["xmin", "ymin", "xmax", "ymax"]  
].values
```

```
y = df["class"].values
```

```
# -----  
# Encode Classes  
# -----
```

```
encoder = LabelEncoder()
```

```
y = encoder.fit_transform(y)
```

```
print("\nClasses:")  
print(encoder.classes_)
```

```
# -----  
# Split Dataset  
# -----
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.20,
```

```
    random_state=42,
    stratify=y
)

# -----
# Normalize Coordinates
# -----

X_train = X_train / 640.0
X_test = X_test / 640.0

# -----
# Build Neural Network
# -----

model = Sequential()

model.add(
    Dense(
        64,
        activation="relu",
        input_shape=(4,)
    )
)

model.add(
    Dense(
        128,
        activation="relu"
    )
)

model.add(
    Dropout(0.2)
)

model.add(
    Dense(
        64,
        activation="relu"
    )
)

model.add(
    Dense(
        32,
        activation="relu"
    )
)
```

```

# Output:
# Car, Pedestrian, Road Sign

model.add(
    Dense(
        3,
        activation="softmax"
    )
)

# -----
# Compile
# -----

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()

# -----
# Train
# -----

history = model.fit(
    X_train,
    y_train,
    epochs=30,
    batch_size=32,
    validation_split=0.20,
    verbose=1
)

# -----
# Test
# -----

loss, accuracy = model.evaluate(
    X_test,
    y_test,
    verbose=0
)

print(
    "\nTest Accuracy:",
    round(accuracy * 100, 2),
    "%"
)

```

```
# =====  
# GRAPH 1 — ACCURACY  
# =====
```

```
plt.figure(figsize=(8,5))  
  
plt.plot(  
    history.history["accuracy"],  
    label="Training Accuracy"  
)  
  
plt.plot(  
    history.history["val_accuracy"],  
    label="Validation Accuracy"  
)  
  
plt.title(  
    "Autonomous Vehicle Object Classification Accuracy"  
)  
  
plt.xlabel("Epoch")  
plt.ylabel("Accuracy")  
  
plt.legend()  
plt.grid()  
  
plt.show()
```

```
# =====  
# GRAPH 2 — LOSS  
# =====
```

```
plt.figure(figsize=(8,5))  
  
plt.plot(  
    history.history["loss"],  
    label="Training Loss"  
)  
  
plt.plot(  
    history.history["val_loss"],  
    label="Validation Loss"  
)  
  
plt.title(  
    "Autonomous Vehicle Model Loss"  
)  
  
plt.xlabel("Epoch")
```

```

plt.ylabel("Loss")

plt.legend()
plt.grid()

plt.show()

# =====
# PREDICTION
# =====

predictions = model.predict(X_test)

predicted_classes = np.argmax(
    predictions,
    axis=1
)

# -----
# Display Sample Predictions
# -----

for i in range(10):

    actual = encoder.inverse_transform(
        [y_test[i]]
    )[0]

    predicted = encoder.inverse_transform(
        [predicted_classes[i]]
    )[0]

    confidence = np.max(
        predictions[i]
    ) * 100

    print(
        "Actual:",
        actual,
        "| Predicted:",
        predicted,
        "| Confidence:",
        round(confidence, 2),
        "%"
    )

```



```

[...][...] autonomous_vehicle_object_detection.csv
autonomous_vehicle_object_detection.csv(text/csv) - 91243 bytes, last modified: 8/10/2026 - 100% done
Saving autonomous_vehicle_object_detection.csv to autonomous_vehicle_object_detection.csv
... Dataset Shape: (2433, 7)

First 5 Rows:
   image_id  object_id  class  xmin  ymin  xmax  ymax
0  img_0001.jpg        1  pedestrian  270   106   371   196
1  img_0001.jpg        2  pedestrian  121   214   225   318
2  img_0001.jpg        3   road_sign   99   359   152   391
3  img_0002.jpg        1        car   343   293   374   386
4  img_0002.jpg        2   road_sign  160   313   211   450

Classes:
['car' 'pedestrian' 'road_sign']
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential:
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_2"

```

Layer (type)	Output Shape	Param #
dense_8 (Dense)	(None, 64)	320
dense_9 (Dense)	(None, 128)	8,320
dropout (Dropout)	(None, 128)	0
dense_10 (Dense)	(None, 64)	8,256
dense_11 (Dense)	(None, 32)	2,080
dense_12 (Dense)	(None, 3)	99

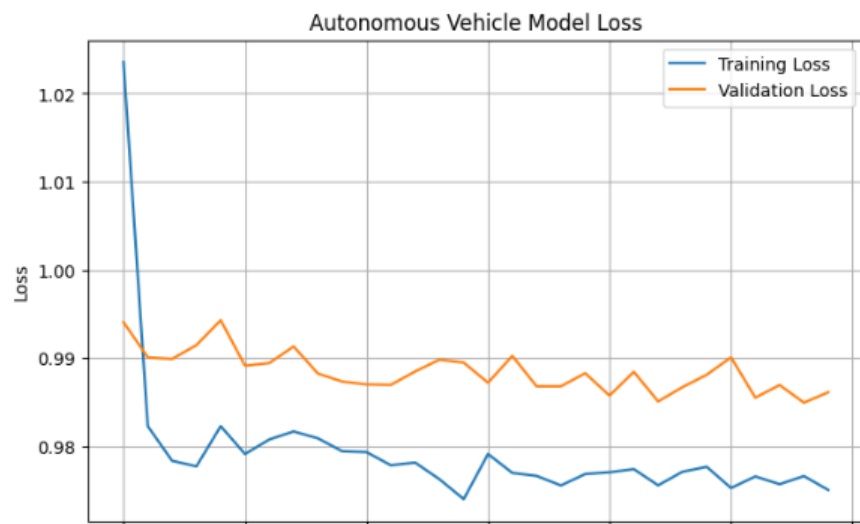
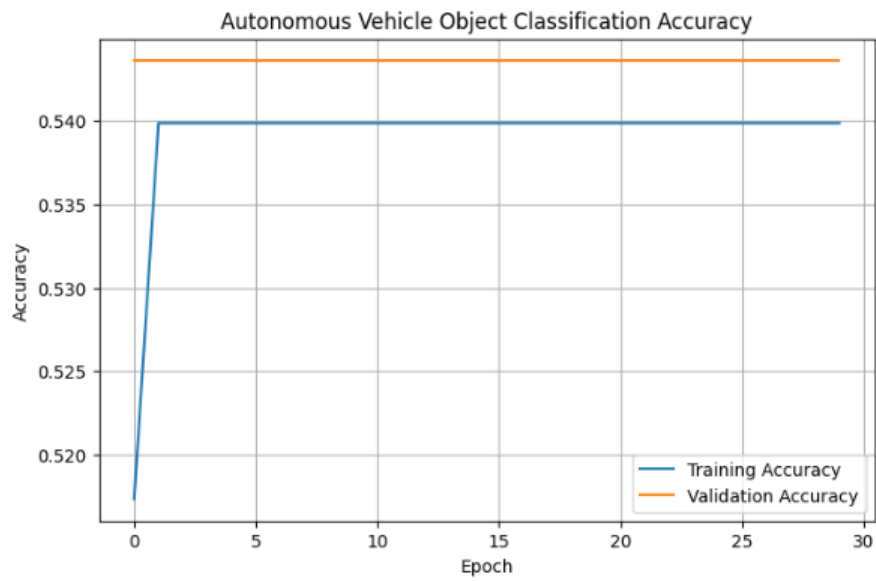
```

Total params: 19,075 (74.51 KB)
Trainable params: 19,075 (74.51 KB)
Non-trainable params: 0 (0.00 B)
Epoch 1/30
49/49 — 2s 8ms/step - accuracy: 0.5174 - loss: 1.0236 - val_accuracy: 0.5436 - val_loss: 0.9941
Epoch 2/30
49/49 — 0s 5ms/step - accuracy: 0.5398 - loss: 0.9823 - val_accuracy: 0.5436 - val_loss: 0.9901
Epoch 3/30
49/49 — 0s 4ms/step - accuracy: 0.5398 - loss: 0.9784 - val_accuracy: 0.5436 - val_loss: 0.9899
Epoch 4/30
49/49 — 0s 4ms/step - accuracy: 0.5398 - loss: 0.9777 - val_accuracy: 0.5436 - val_loss: 0.9915
Epoch 5/30
49/49 — 0s 4ms/step - accuracy: 0.5398 - loss: 0.9823 - val_accuracy: 0.5436 - val_loss: 0.9943
Epoch 6/30
49/49 — 0s 4ms/step - accuracy: 0.5398 - loss: 0.9791 - val_accuracy: 0.5436 - val_loss: 0.9891
Epoch 7/30
49/49 — 0s 5ms/step - accuracy: 0.5398 - loss: 0.9808 - val_accuracy: 0.5436 - val_loss: 0.9894
Epoch 8/30
49/49 — 0s 4ms/step - accuracy: 0.5398 - loss: 0.9817 - val_accuracy: 0.5436 - val_loss: 0.9913
Epoch 9/30
49/49 — 0s 5ms/step - accuracy: 0.5398 - loss: 0.9809 - val_accuracy: 0.5436 - val_loss: 0.9883

```

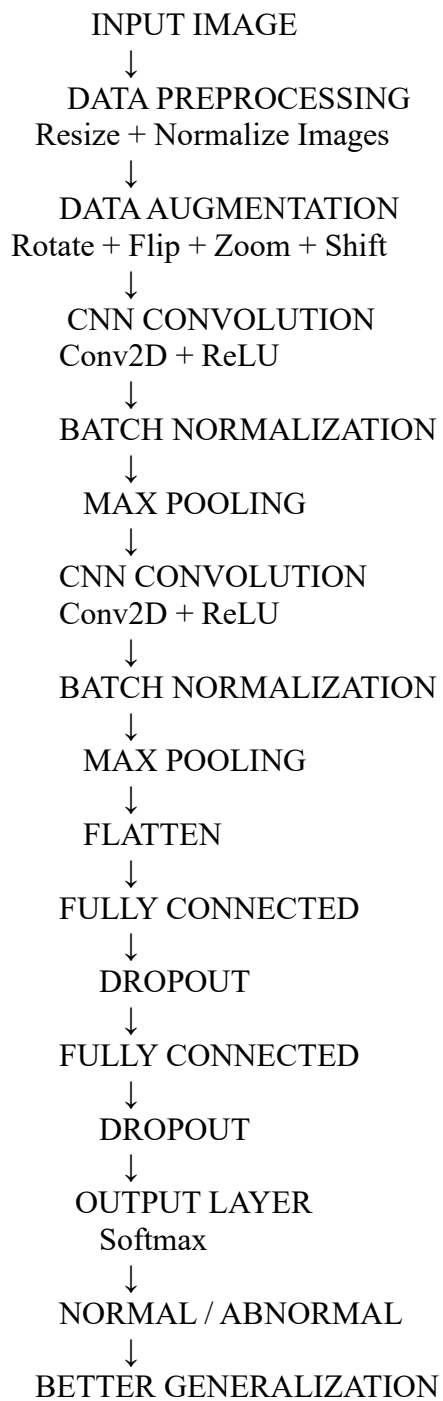
Test Accuracy: 54.0 %

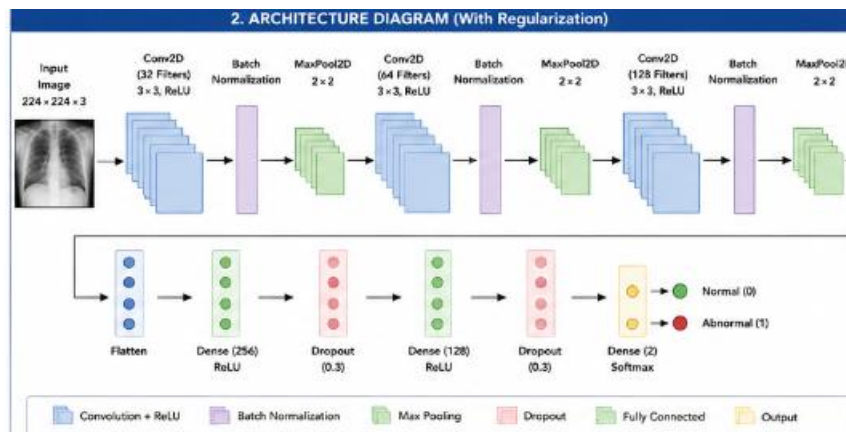
...



4. Case Study: CNN Overfitting

Workflow





Dataset:

```

xray_normal_abnormal_dataset.csv  autonomous_vehicle_object_detection.csv  cnn_overfitting_dataset.csv  face_recognition_features.csv

C: > Users > SAMANVI > Downloads > saveetha clg > DL > CO3 > AT1 > cnn_overfitting_dataset.csv

1 pixel_1,pixel_2,pixel_3,pixel_4,pixel_5,pixel_6,pixel_7,pixel_8,pixel_9,pixel_10,pixel_11,pixel_12,pixel_13,pixel_14,pixel_15
2 0.1106511423692456,0.089668886913327608,0.126523222726712,0.16783870596373673,0.10449946300485985,0.1117355270031365,0.19
3 0.11317419416606342,0.13843067020276106,0.13397166693684925,0.12528471494374244,0.11105897009801619,0.0546625716088734,0.
4 0.126166847843304,0.05843223563129481,0.05619880005680265,0.12959973240293984,0.060539305567647084,0.217692645227449,0.13
5 0.14627287269135283,0.14106439784746774,0.053196286645591825,0.12775623859614754,0.17467916883570575,0.10130977278793248,
6 0.08691762944820071,0.07856076406222913,0.06278493832077224,0.13124735937481447,0.061180321195929886,0.15214212183484277,
7 0.17814649622484313,0.11802005973398766,0.07739052564041288,0.08246873445151748,0.1101682596976429,0.11156993734666894,0.
8 0.09851668743272889,0.06915532529956066,0.13134814044025608,0.08194158049727257,0.0,0.12624712319927242,0.106032012277683
9 0.08069953761239777,0.1220808390063446,0.08380403652502999,0.11134644907499007,0.13153074393100322,0.10439676657164558,0.
10 0.012888877283163718,0.1343377534067855,0.06819363555577843,0.14119653318672717,0.11112773828713665,0.12225202784434368,0.
11 0.09290494876675975,0.12981103142887862,0.08153972233612042,0.174002146555643,0.055445562114814176,0.16875143272901946,0.
12 0.15914113832999202,0.11596222275501229,0.11250415164854413,0.08832457750233168,0.11030909435765479,0.0935520390201328,0.
13 0.11903744357793725,0.09184986405320993,0.08863087494805358,0.17432889217837205,0.1850022208499167,0.14671693568083533,0.
14 0.08693408576717314,0.10555665830915106,0.12208707951763963,0.056934915230455525,0.17121094234650838,0.046714483659111794
15 0.03748458500887154,0.12045618543112074,0.13257750367663457,0.10716276976557715,0.09154026817006404,0.16087616056897008,0.
16 0.08150561616425138,0.08800919585706717,0.08127601994954348,0.13436237605837054,0.08850585946353272,0.10670226842413875,0.
17 0.09069622115323436,0.028398616323981673,0.11683704390636733,0.17075464605185456,0.08559814489240151,0.08856137933318893,
18 0.11263853431548716,0.08085774007761908,0.0426082540671754,0.09887596083275033,0.1044307394124462,0.16305278286795338,0.1
19 0.06582896630206558,0.08412107742832789,0.14160573308173496,0.14327501581393784,0.07398323038656425,0.07881132766081177,0.
20 0.11447620704165191,0.0640881546319475,0.0,0.05757815531061068,0.13220321406792518,0.1519944468928761,0.19690086883427577
21 0.15039445191556822,0.04868362853021142,0.13713622013449397,0.12233391878469764,0.10534198191644684,0.0773358931295549,0.
22 0.1592748853834732,0.09005739980471827,0.06161392535110291,0.13722770291574304,0.09600741886359136,0.11783153823984395,0.
23 0.12502292445729665,0.07625670741110738,0.06827555177260855,0.08319795106628755,0.05766414337393786,0.12575023902403087,0.
24 0.08335519640365048,0.03251757826788322,0.10715227363720918,0.13856052946231773,0.08664487510245669,0.0845542573832542,0.
25 0.0,0.10763195062212168,0.10487544380159981,0.11442690431823709,0.1334370969477857,0.061922871957514114,0.095922483219778
26 0.0926981275558049,0.08393190834855828,0.08962148562635439,0.1217375921405394,0.1359833255617175,0.09537106803260105,0.09
27 0.06270003447693703,0.15530855446028244,0.04157997630371607,0.11455290943256376,0.10862157161973604,0.10737440254485008,0.
28 0.062468928818586525,0.15121167347509462,0.096329372417454,0.13083862132064086,0.09060713227517088,0.15498178836692197,0.
29 0.14046738975158232,0.06744591162496337,0.11811708483527097,0.05497618182168611,0.15729181390654343,0.17792794212100116,0.
30 0.1068338728958845,0.10067836476407017,0.15958729832031254,0.11281440936432022,0.12264845223943635,0.1263700168037098,0.1
31 0.09712524541707306,0.0707811158885165,0.18693555762360137,0.06461239393714072,0.16989153915006114,0.1298912306928426,0.1
32 0.0976265597409913,0.1302834812811618,0.1486322554155648,0.13277547079692828,0.14323909672442206,0.09949725703096803,0.07
33 0.0964571081770815,0.08305808091141353,0.0388901065561975,0.06678593206358796,0.11405423643048493,0.12212698724866765,0.1
34 0.14689314867202427,0.1226976970952446,0.12667270278348777,0.04311622302916712,0.09004599829422182,0.10771916412415881,0.
35 0.026665619721156702,0.0830713977269818,0.09496105252965589,0.05961671081833299,0.10907736867282403,0.1270208296776968,0.

```

Program:

```

# =====
# CNN OVERFITTING - SOLUTION
# Dropout + Batch Normalization
# + Data Augmentation
# =====

```

!pip -q install tensorflow

```

# -----
# Import Libraries
# -----

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from google.colab import files

from sklearn.model_selection import train_test_split

import tensorflow as tf

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import (
    Conv2D,
    MaxPooling2D,
    BatchNormalization,
    Flatten,
    Dense,
    Dropout
)

from tensorflow.keras.preprocessing.image import ImageDataGenerator

# -----
# Upload Dataset
# -----

uploaded = files.upload()

# -----
# Load Dataset
# -----

df = pd.read_csv("cnn_overfitting_dataset.csv")

print("Dataset Shape:", df.shape)

print(df.head())

# -----
# Separate Features and Labels
# -----

X = df.iloc[:, :-1].values

y = df["label"].values

```

```

# -----
# Reshape into 28 × 28 Images
# -----

X = X.reshape(-1, 28, 28, 1)

# -----
# Train-Test Split
# -----

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training Samples:", len(X_train))
print("Testing Samples:", len(X_test))

# =====
# DATA AUGMENTATION
# =====

datagen = ImageDataGenerator(
    rotation_range=15,
    width_shift_range=0.10,
    height_shift_range=0.10,
    zoom_range=0.10,
    horizontal_flip=True
)

datagen.fit(X_train)

# =====
# BUILD CNN WITH REGULARIZATION
# =====

model = Sequential()

# -----
# CNN BLOCK 1
# -----

model.add(
    Conv2D(
        32,
        (3,3),

```

```

        activation="relu",
        padding="same",
        input_shape=(28,28,1)
    )
)

# Batch Normalization
model.add(
    BatchNormalization()
)

# Pooling
model.add(
    MaxPooling2D((2,2))
)

# -----
# CNN BLOCK 2
# -----

model.add(
    Conv2D(
        64,
        (3,3),
        activation="relu",
        padding="same"
    )
)

# Batch Normalization
model.add(
    BatchNormalization()
)

# Pooling
model.add(
    MaxPooling2D((2,2))
)

# -----
# CNN BLOCK 3
# -----

model.add(
    Conv2D(
        128,
        (3,3),
        activation="relu",
        padding="same"
    )
)

```

```
)

# Batch Normalization
model.add(
    BatchNormalization()
)

# Pooling
model.add(
    MaxPooling2D((2,2))
)

# -----
# Flatten
# -----

model.add(
    Flatten()
)

# -----
# Fully Connected Layer
# -----

model.add(
    Dense(
        128,
        activation="relu"
    )
)

# -----
# DROPOUT
# -----

model.add(
    Dropout(0.5)
)

# -----
# Output
# -----

model.add(
    Dense(
        1,
        activation="sigmoid"
    )
)
```



```

# =====
# COMPILE MODEL
# =====

model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

# -----
# Display Architecture
# -----

model.summary()

# =====
# TRAIN MODEL
# =====

history = model.fit(
    datagen.flow(
        X_train,
        y_train,
        batch_size=32
    ),
    epochs=20,
    validation_data=(X_test, y_test)
)

# =====
# EVALUATION
# =====

loss, accuracy = model.evaluate(
    X_test,
    y_test,
    verbose=0
)

print("\nFinal Test Accuracy:",
      round(accuracy * 100, 2), "%")

print("Final Test Loss:",
      round(loss, 4))

# =====
# GRAPH 1
# ACCURACY
# =====

```

```

plt.figure(figsize=(8,5))

plt.plot(
    history.history["accuracy"],
    label="Training Accuracy"
)

plt.plot(
    history.history["val_accuracy"],
    label="Validation Accuracy"
)

plt.title(
    "CNN Accuracy After Reducing Overfitting"
)

plt.xlabel("Epoch")

plt.ylabel("Accuracy")

plt.legend()

plt.grid()

plt.show()

# =====
# GRAPH 2
# LOSS
# =====

plt.figure(figsize=(8,5))

plt.plot(
    history.history["loss"],
    label="Training Loss"
)

plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
)

plt.title(
    "CNN Loss After Regularization"
)

plt.xlabel("Epoch")

```

```
plt.ylabel("Loss")

plt.legend()

plt.grid()

plt.show()

# =====
# PREDICTION
# =====

predictions = model.predict(X_test)

predictions = (
    predictions > 0.5
).astype(int)

print("\nSample Predictions:")

for i in range(10):

    actual = int(y_test[i])

    predicted = int(predictions[i][0])

    print(
        "Actual:",
        actual,
        "Predicted:",
        predicted
    )
```

```

    pixel_8 pixel_9 pixel_10 ... pixel_776 pixel_777 pixel_778 \
0 0.165415 0.121534 0.166468 ... 0.091667 0.118481 0.129612
1 0.143025 0.151176 0.093697 ... 0.143631 0.146712 0.109624
2 0.148633 0.107976 0.132155 ... 0.148414 0.119420 0.176792
3 0.113596 0.120030 0.078028 ... 0.059668 0.157654 0.092070
4 0.104620 0.081641 0.124710 ... 0.058864 0.131078 0.176052

```

```

    pixel_779 pixel_780 pixel_781 pixel_782 pixel_783 pixel_784 label
0 0.127776 0.146645 0.120347 0.052042 0.113977 0.117738 0.0
1 0.033783 0.157180 0.142712 0.094203 0.101448 0.113952 1.0
2 0.203287 0.095521 0.095949 0.098923 0.120411 0.087175 0.0
3 0.155091 0.150391 0.106361 0.098669 0.066514 0.039491 1.0
4 0.152258 0.138946 0.102039 0.054295 0.119381 0.059943 0.0

```

[5 rows x 785 columns]

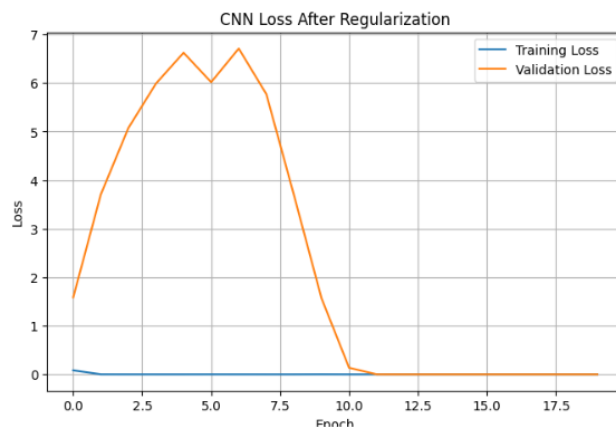
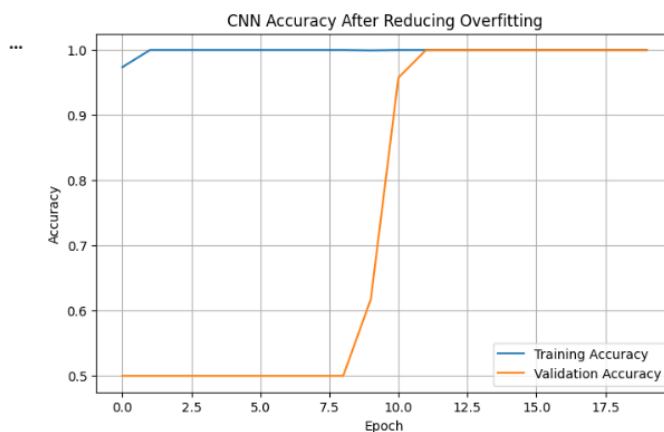
Training Samples: 1600

Testing Samples: 400

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not call `super().__init__(activity_regularizer=activity_regularizer, **kwargs)` with `activity_regularizer` as it is deprecated. Please use `kernel_regularizer` or `bias_regularizer` instead.

Model: "sequential_3"

Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 28, 28, 32)	320
batch_normalization (BatchNormalization)	(None, 28, 28, 32)	128
max_pooling2d_3 (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_4 (Conv2D)	(None, 14, 14, 64)	18,496
batch_normalization_1 (BatchNormalization)	(None, 14, 14, 64)	256
max_pooling2d_4 (MaxPooling2D)	(None, 7, 7, 64)	0
conv2d_5 (Conv2D)	(None, 7, 7, 128)	73,856
batch_normalization_2 (BatchNormalization)	(None, 7, 7, 128)	512
max_pooling2d_5 (MaxPooling2D)	(None, 3, 3, 128)	0
flatten_1 (Flatten)	(None, 1152)	0



Question 5: Case Study – Image Classification

Workflow

Fruit Image



Image Preprocessing
(Resize + Normalize)



Convolution Layer 1
(32 Filters + ReLU)



Max Pooling



Convolution Layer 2
(64 Filters + ReLU)



Max Pooling



Convolution Layer 3
(128 Filters + ReLU)



Max Pooling



Flatten



Fully Connected Layer
(128 Neurons + ReLU)



Dropout



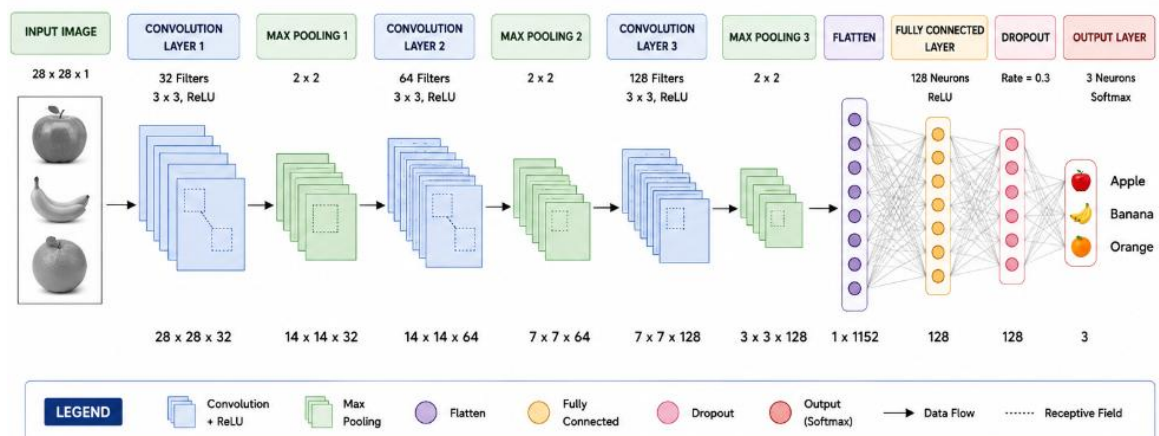
Output Layer
(Softmax)



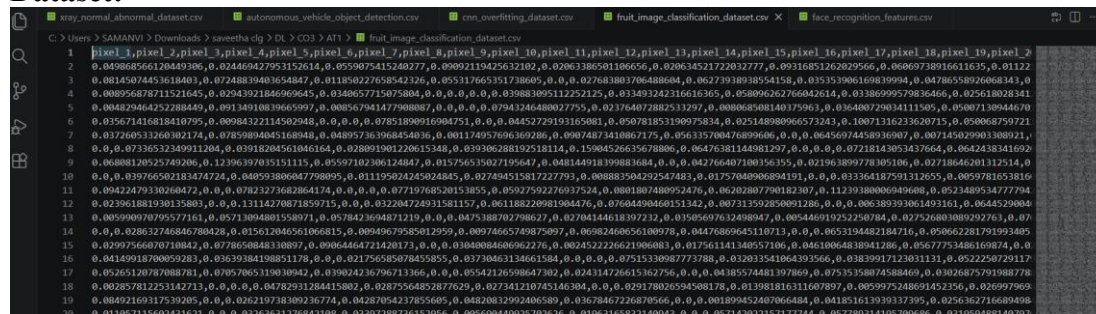
Predicted Fruit



Apple / Banana / Orange



Dataset:



Program:

```
# =====  
# FRUIT IMAGE CLASSIFICATION USING CNN  
# =====
```

!pip -q install tensorflow

```
# -----  
# Import Libraries  
# -----
```

```
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt
```

```
from google.colab import files
```

```
from sklearn.model_selection import train_test_split  
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
```

```
import tensorflow as tf
```

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import (  
    Conv2D,  
    MaxPooling2D,  
    Flatten,  
    Dense,  
    Dropout  
)
```

```
# -----  
# Upload Dataset  
# -----
```

```
uploaded = files.upload()
```

```

# -----
# Load Dataset
# -----

df = pd.read_csv(
    "fruit_image_classification_dataset.csv"
)

print("Dataset Shape:", df.shape)

print("\nFirst 5 Rows:")
print(df.head())

# -----
# Separate Features and Labels
# -----

X = df.iloc[:, :-1].values
y = df["label"].values.astype(int)

# -----
# Reshape Pixels into Images
# -----

X = X.reshape(-1, 28, 28, 1)

# -----
# Train-Test Split
# -----

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# -----
# Build CNN
# -----

model = Sequential()

# Convolution Block 1

model.add(
    Conv2D(
        32,

```

```

        (3,3),
        activation="relu",
        input_shape=(28,28,1)
    )
)

model.add(
    MaxPooling2D((2,2))
)

# -----
# Convolution Block 2
# -----

model.add(
    Conv2D(
        64,
        (3,3),
        activation="relu"
    )
)

model.add(
    MaxPooling2D((2,2))
)

# -----
# Convolution Block 3
# -----

model.add(
    Conv2D(
        128,
        (3,3),
        activation="relu"
    )
)

model.add(
    MaxPooling2D((2,2))
)

# -----
# Flatten
# -----

model.add(
    Flatten()
)

```



```

# -----
# Fully Connected Layer
# -----

model.add(
    Dense(
        128,
        activation="relu"
    )
)

# -----
# Dropout
# -----

model.add(
    Dropout(0.3)
)

# -----
# Output Layer
# -----

model.add(
    Dense(
        3,
        activation="softmax"
    )
)

# -----
# Compile
# -----

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

# -----
# Display Architecture
# -----

model.summary()

# -----
# Train CNN
# -----

```

```

history = model.fit(
    X_train,
    y_train,
    epochs=15,
    batch_size=32,
    validation_split=0.20,
    verbose=1
)

# -----
# Test Model
# -----

loss, accuracy = model.evaluate(
    X_test,
    y_test,
    verbose=0
)

print("\nTest Accuracy:",
      round(accuracy * 100, 2), "%")

print("Test Loss:",
      round(loss, 4))

# =====
# ACCURACY GRAPH
# =====

plt.figure(figsize=(8,5))

plt.plot(
    history.history["accuracy"],
    label="Training Accuracy"
)

plt.plot(
    history.history["val_accuracy"],
    label="Validation Accuracy"
)

plt.title(
    "Fruit Classification - Model Accuracy"
)

plt.xlabel("Epoch")

plt.ylabel("Accuracy")

plt.legend()

```

```

plt.grid()

plt.show()

# =====
# LOSS GRAPH
# =====

plt.figure(figsize=(8,5))

plt.plot(
    history.history["loss"],
    label="Training Loss"
)

plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
)

plt.title(
    "Fruit Classification - Model Loss"
)

plt.xlabel("Epoch")

plt.ylabel("Loss")

plt.legend()

plt.grid()

plt.show()

# =====
# PREDICTION
# =====

predictions = model.predict(X_test)

predicted_classes = np.argmax(
    predictions,
    axis=1
)

# =====
# CONFUSION MATRIX
# =====

```

```

cm = confusion_matrix(
    y_test,
    predicted_classes
)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=[
        "Apple",
        "Banana",
        "Orange"
    ]
)

disp.plot()

plt.title(
    "Fruit Classification Confusion Matrix"
)

plt.show()

# =====
# SAMPLE PREDICTIONS
# =====

class_names = [
    "Apple",
    "Banana",
    "Orange"
]
print("\nSample Predictions:")
for i in range(10):
    actual = class_names[y_test[i]]
    predicted = class_names[
        predicted_classes[i]
    ]
    confidence = (
        np.max(predictions[i]) * 100
    )
    print(
        "Actual:",
        actual,
        "| Predicted:",
        predicted,
        "| Confidence:",
        round(confidence, 2),
        "%"
    )

```

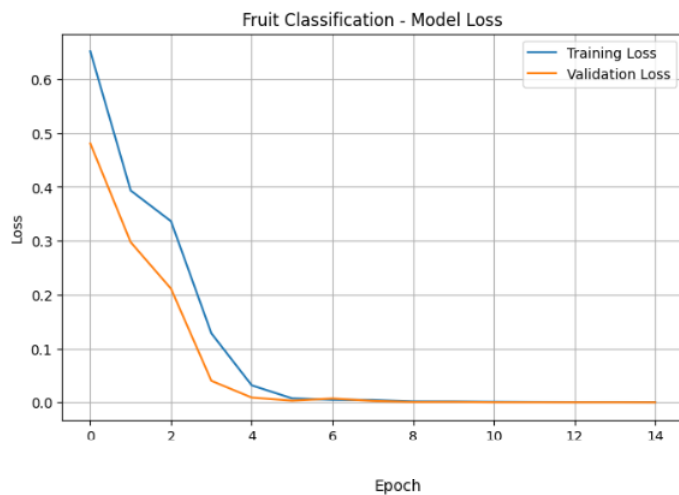
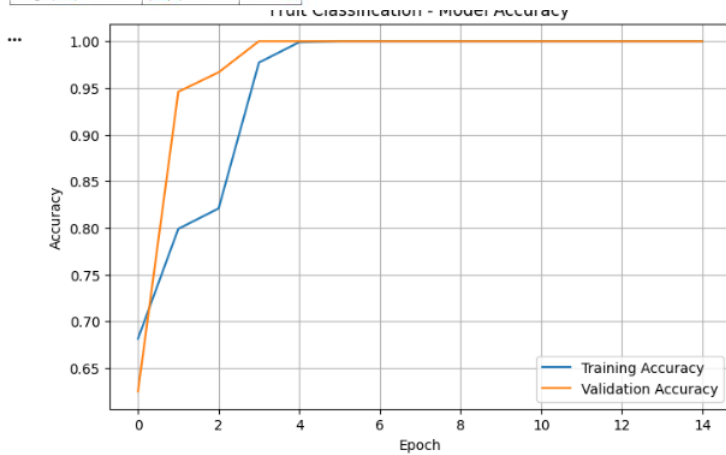
```

pixel_8 pixel_9 pixel_10 ... pixel_776 pixel_777 pixel_778 \
0 0.000007 0.011121 0.011902 ... 0.000000 0.000010 0.020000
1 0.005354 0.047866 0.021469 ... 0.007862 0.044129 0.034060
2 0.013097 0.000004 0.013078 ... 0.000004 0.014461 0.107308
3 0.000069 0.016461 0.090071 ... 0.019294 0.003871 0.000000
4 0.000782 0.025149 0.100713 ... 0.051969 0.091327 0.070152

pixel_779 pixel_780 pixel_781 pixel_782 pixel_783 pixel_784 label
0 0.027021 0.054827 0.037188 0.000000 0.005206 0.054423 0.0
1 0.000000 0.000000 0.044201 0.021155 0.000000 0.000000 0.0
2 0.012778 0.004485 0.000000 0.000000 0.070991 0.000000 0.0
3 0.020636 0.005887 0.047201 0.040882 0.001695 0.001513 0.0
4 0.035449 0.111774 0.004067 0.025988 0.018782 0.025983 0.0

[5 rows x 785 columns]
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an 'input_shape' argument to a layer. When using Sequential models, prefer using an 'Input(shape)' object as the first layer in the model instead.
  super().__init__(activity_regularizer=regularizer, **kwargs)
Model: "sequential_4"
```

Layer (type)	Output Shape	Param #
conv2d_6 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_5 (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_7 (Conv2D)	(None, 11, 11, 64)	35,456
max_pooling2d_7 (MaxPooling2D)	(None, 5, 5, 64)	0
conv2d_8 (Conv2D)	(None, 5, 5, 128)	75,856
max_pooling2d_8 (MaxPooling2D)	(None, 1, 1, 128)	0
flatten_1 (Flatten)	(None, 128)	0
dense_15 (Dense)	(None, 128)	16,512
dropout_2 (Dropout)	(None, 128)	0
dense_16 (Dense)	(None, 3)	387



10/10 — 0s 21ms/step

